Experiment No 3
LAB MANUAL
PART B

Roll No. C-60	**Name: Avdhut Parish Madhavi**
Class: SE-C	**Batch: C2**
Date of Experiment:	**Date of Submission:**
Grade:	

B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

- Compare the number of comparisons made by the Divide and Conquer Min-Max program against a simple linear scan that tracks min and max in a single pass, for the same array.

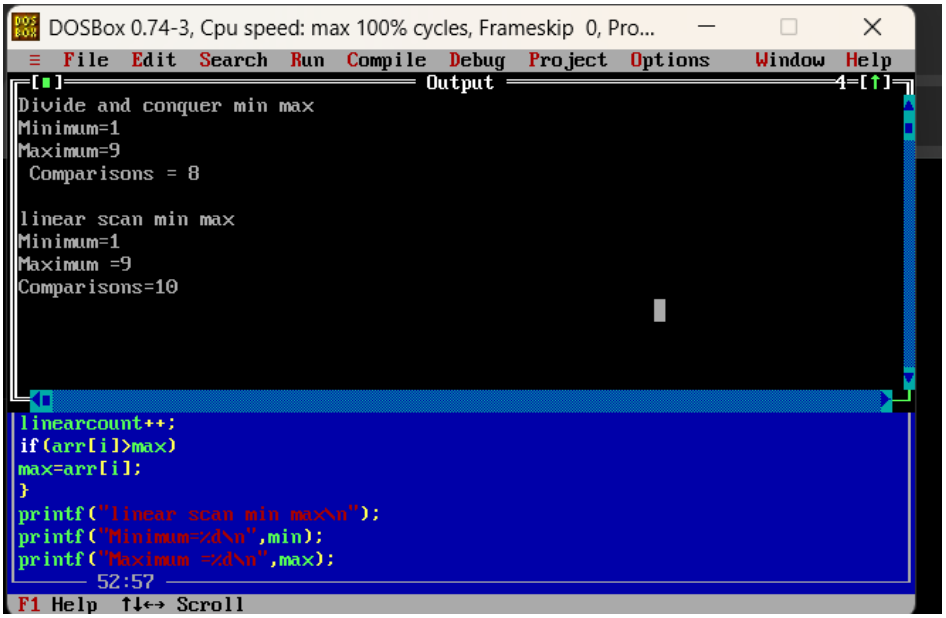
  CODE:

```
#include <stdio.h>
#include <conio.h>
int dcCount = 0;
struct Pair
{ int min;
  int max;
};
struct Pair maxMin(int arr[], int low, int high)
{
  struct Pair result, left, right;
  int mid;
  if (low == high)
  {
    result.min = arr[low];
    result.max = arr[low];
    return result;
  }
  if (high == low + 1)
  {
    dcCount++;
    if (arr[low] < arr[high])
    {
      result.min = arr[low];
```

```
            result.max = arr[high];
        }
        else
        {
            result.min = arr[high];
            result.max = arr[low];
        }
        return result;
    }

    mid = (low + high) / 2;
    left = maxMin(arr, low, mid);
    right = maxMin(arr, mid + 1, high);
    dcCount++;
    result.min = (left.min < right.min) ?
                left.min : right.min;
    dcCount++;
    result.max = (left.max > right.max) ?
                left.max : right.max;
    return result;
}
int main()
{
    int arr[6] = {8, 3, 5, 1, 9, 2};
    int n = 6;
    int i, min, max;
    int linearCount = 0;
    struct Pair result;
    clrscr();
    result = maxMin(arr, 0, n - 1);
    printf("Divide and Conquer Min-Max\n");
    printf("Minimum = %d\n", result.min);
    printf("Maximum = %d\n", result.max);
    printf("Comparisons = %d\n\n", dcCount);
    min = arr[0];
    max = arr[0];
```

```
for (i = 1; i < n; i++)
{
   linearCount++;
   if (arr[i] < min)
      min = arr[i];
   linearCount++;
   if (arr[i] > max)
      max = arr[i];
}
printf("Linear Scan Min-Max\n");
printf("Minimum = %d\n", min);
printf("Maximum = %d\n", max);
printf("Comparisons = %d\n", linearCount);
getch();
return 0;   }
```



- Modify the Quick Sort program to choose the first element as the pivot instead of the last, and compare its performance with the original on the same reverse-sorted array.

CODE:

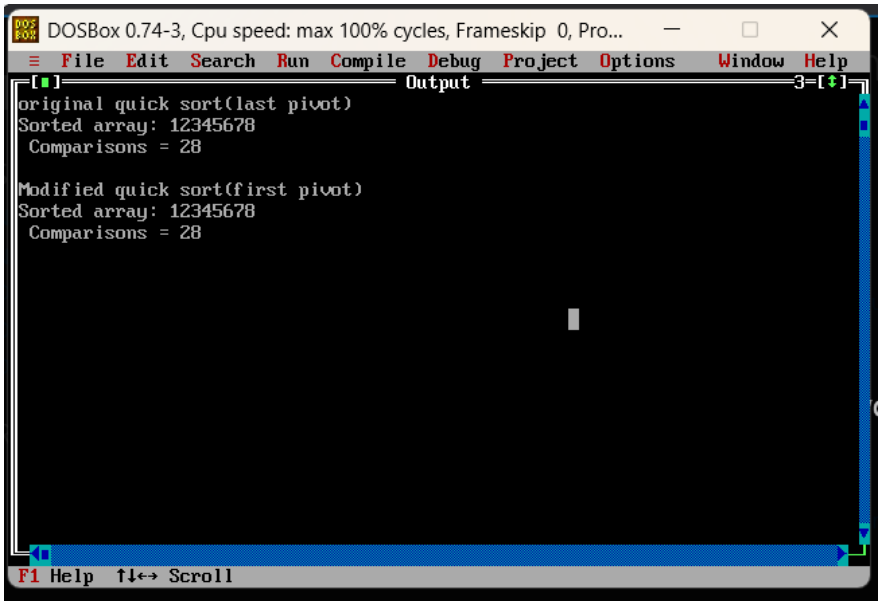
```
#include <stdio.h>
#include <conio.h>
int lastCount = 0;
int firstCount = 0;
```

```
void swap(int *a, int *b)
{
   int temp;
   temp = *a;
   *a = *b;
   *b = temp;
}
int partitionLast(int arr[], int low, int high)
{
   int pivot, i, j;
   pivot = arr[high];
   i = low - 1;
   for (j = low; j < high; j++)
   {
      lastCount++;
      if (arr[j] <= pivot)
      {
         i++;
         swap(&arr[i], &arr[j]);
      }
   }
   swap(&arr[i + 1], &arr[high]);
   return i + 1;
}
void quickSortLast(int arr[], int low, int high)
{
   int pi;
   if (low < high)
   {
      pi = partitionLast(arr, low, high);
      quickSortLast(arr, low, pi - 1);
      quickSortLast(arr, pi + 1, high);
   }
}
int partitionFirst(int arr[], int low, int high)
{
   int pivot, i, j;
```

```
    swap(&arr[low], &arr[high]);
    pivot = arr[high];
    i = low - 1;
    for (j = low; j < high; j++)
    {
        firstCount++;
        if (arr[j] <= pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSortFirst(int arr[], int low, int high)
{
    int pi;
    if (low < high)
    {
        pi = partitionFirst(arr, low, high);
        quickSortFirst(arr, low, pi - 1);
        quickSortFirst(arr, pi + 1, high);
    }
}
void main()
{
    int a[8] = {8, 7, 6, 5, 4, 3, 2, 1};
    int b[8] = {8, 7, 6, 5, 4, 3, 2, 1};
    int i;
    clrscr();
    quickSortLast(a, 0, 7);
    quickSortFirst(b, 0, 7);
    printf("Original Quick Sort (Last Pivot)\n");
    printf("Sorted array: ");
    for (i = 0; i < 8; i++)
```

```
    printf("%d ", a[i]);
  printf("\nComparisons = %d\n\n", lastCount);
  printf("Modified Quick Sort (First Pivot)\n");
  printf("Sorted array: ");
  for (i = 0; i < 8; i++)
    printf("%d ", b[i]);
  printf("\nComparisons = %d", firstCount);
  getch();
}
```



B.2: Write an observation according to the program execution.

Both programs sort the reverse-sorted array correctly. Both perform 28 comparisons because each pivot creates highly unbalanced partitions. Therefore, both have worst-case time complexity of (O(n2)) for this input.